

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
бюджетное образовательное учреждение
высшего образования
«Московский технический университет связи и информатики»
(МТУСИ)

ОТЧЁТ

по производственной практике (проектно-технологической)

Выполнил:

Студент группы БПИ2304

Жуков Никита Александрович

(дата, подпись)

Проверил руководитель практики:

Жариков Александр Романович

Руководитель отдела аналитики и архитектуры

(дата, подпись)

Москва, 2026

Оглавление

Введение	3
1 Теоретические основы омниканальных решений	5
1.1 Понятие омниканальности	5
1.2 Эволюция каналов взаимодействия с клиентами	5
1.3 Ключевые компоненты омниканальной архитектуры	6
1.4 Преимущества внедрения омниканальных решений	7
1.5 Технологические аспекты реализации	7
1.6 Отраслевые особенности применения	8
2 Архитектура программного комплекса омниканального решения	9
2.1 Общая архитектура системы	9
2.2 Серверная часть (Backend)	9
2.2.1 Архитектурный подход	9
2.2.2 Схема слоёв бэкенда	10
2.2.3 Технологический стек бэкенда	10
2.2.4 API-эндпоинты	10
2.3 Клиентская часть (Frontend)	11
2.3.1 Структура пользовательского интерфейса	11
2.3.2 Взаимодействие с сервером	11
2.4 Схема базы данных	12
2.5 Модуль интеграции каналов связи	12
2.5.1 Модуль Telegram	13
2.5.2 Модуль Email	14
2.5.3 Планируемые каналы	14
2.6 Взаимодействие компонентов	14
2.6.1 Получение входящего сообщения	14
2.6.2 Отправка ответа	15
2.6.3 Назначение куратора	15

3	Реализация программного комплекса	17
3.1	Доменная модель	17
3.2	Порты (интерфейсы)	18
3.3	Бизнес-логика	19
3.3.1	Сервис сообщений	19
3.3.2	Оркестратор опроса каналов	20
3.4	Адаптеры каналов связи	21
3.4.1	Telegram	21
3.4.2	Email	23
3.4.3	Фабрика движков	23
3.5	Репозиторий и ORM	24
3.6	Контейнер внедрения зависимостей	25
3.7	Конфигурация и развёртывание	26
3.8	Тестирование	27
	Заключение	30
	Приложение А. Исходный код проекта	31

Введение

В современных условиях организации взаимодействия с клиентами и обучающимися сталкиваются с проблемой фрагментации каналов коммуникации. Электронная почта, мессенджеры (Telegram, WhatsApp), образовательные платформы и социальные сети генерируют потоки обращений, которые сложно обрабатывать централизованно. Мониторинг всех этих каналов по отдельности занимает значительное время, и существует высокий риск упустить важное сообщение. В связи с этим разработка «единого окна» коммуникации, объединяющего все каналы в общем интерфейсе оператора, является актуальной задачей в области программной инженерии.

Производственная (проектно-технологическая) практика была направлена на закрепление теоретических знаний, полученных в процессе обучения, и приобретение практических навыков проектирования, разработки и развёртывания современных веб-приложений. Практика проходила в **ООО «ВИЖНЛАБС»**, где передо мной стояла задача создания омниканального решения для центра обучения компании.

В рамках тестового задания требовалось исследовать существующие на рынке омниканальные платформы (Front, respond.io, Zendesk), сформулировать требования к «единому окну» коммуникации для центра обучения, а затем разработать собственное программное решение, позволяющее принимать и обрабатывать обращения из различных каналов в едином интерфейсе куратора. За основу было принято решение разрабатывать самописное решение, а не использовать готовую SaaS-платформу, поскольку это позволяет точно подстроить систему под реальные процессы центра обучения, особенности используемой образовательной платформы, внутренние роли кураторов и правила обработки обращений, а также сохранить полный контроль над данными и логикой распределения.

Целью практики являлось участие в проектировании и разработке омниканальной платформы для центра обучения, обеспечивающей приём и обработку обращений из различных каналов связи в едином интерфейсе куратора.

В ходе выполнения практики были поставлены и решены следующие **задачи**:

1. Изучить теоретические основы омниканальных решений, проанализировать отличия омниканального подхода от мультиканального.
2. Спроектировать архитектуру системы, обеспечивающую модульность, расширяемость

и слабую связанность компонентов.

3. Реализовать серверную часть приложения с использованием современных технологий (Python, Litestar, PostgreSQL).
4. Разработать клиентский интерфейс оператора на базе веб-технологий (React, TypeScript, WebSocket).
5. Интегрировать каналы связи (Telegram, Email) через единый интерфейс движка сообщений.
6. Развернуть систему с использованием контейнеризации Docker и провести тестирование ключевых сценариев работы.

Выполнение данных задач позволило получить практические навыки проектирования архитектуры программных систем, разработки серверных приложений на языке Python с использованием асинхронного веб-фреймворка Litestar, работы с реляционными базами данных PostgreSQL и объектным хранилищем MinIO, интеграции внешних API (Telegram Bot API, IMAP/SMTP), а также контейнеризации и развёртывания приложений с использованием Docker.

Настоящий отчёт содержит описание выполненных в ходе практики работ, принятых архитектурных решений, процесса реализации основных компонентов системы и полученных результатов. Отчёт состоит из введения, трёх глав, заключения и списка литературы. В главе 1 рассмотрены теоретические основы омниканальных решений. В главе 2 описана архитектура разработанного программного комплекса. Глава 3 содержит детали реализации ключевых компонентов системы.

Глава 1

Теоретические основы омниканальных решений

1.1 Понятие омниканальности

В современном мире цифровых технологий и повсеместного распространения интернета взаимодействие компаний с клиентами претерпело кардинальные изменения. Традиционные модели коммуникации, основанные на использовании одного или нескольких изолированных каналов, уступают место более сложным и интегрированным подходам. Одним из таких подходов является омниканальная стратегия, которая предполагает бесшовную интеграцию всех каналов взаимодействия с клиентом в единую экосистему.

Омниканальность (от лат. *omni* — «все» и *channel* — «канал») представляет собой стратегический подход к организации взаимодействия с клиентами, при котором все каналы коммуникации (веб-сайт, мобильное приложение, социальные сети, электронная почта, телефон, физические точки продаж и др.) объединены в единую систему. Ключевой особенностью омниканального подхода является то, что клиент воспринимается как единое целое независимо от того, через какой канал он взаимодействует с компанией, а все каналы обмениваются данными в режиме реального времени.

Важно различать понятия мультиканальности и омниканальности. Мультиканальный подход предполагает использование нескольких каналов коммуникации, однако эти каналы часто существуют изолированно друг от друга. Например, клиент может оформить заказ на веб-сайте, но при обращении в контакт-центр ему придётся заново сообщать всю информацию. В омниканальной модели такая ситуация исключена: история взаимодействия клиента доступна во всех каналах одновременно, что обеспечивает непрерывность клиентского опыта.

1.2 Эволюция каналов взаимодействия с клиентами

Развитие каналов взаимодействия с клиентами прошло несколько этапов. На начальном этапе компании использовали исключительно офлайн-каналы: личные встречи, телефонные звонки, почтовые рассылки. С развитием интернета появились онлайн-каналы: веб-сайты,

электронная почта, чаты. Вслед за этим возникла необходимость интеграции различных каналов, что привело к появлению мультиканального подхода.

Таблица 1.1: Эволюция подходов к взаимодействию с клиентами

Характеристика	Мультиканальный подход	Оmnikanальный подход
Интеграция каналов	Частичная или отсутствует	Полная интеграция
Единая история клиента	Отсутствует	Единая для всех каналов
Качество обслуживания	Различается по каналам	Единое на всех каналах
Аналитика	Разрозненная	Консолидированная

Современный этап развития характеризуется переходом к омниканальности, что обусловлено несколькими факторами. Во-первых, изменилось поведение потребителей: современный клиент использует в среднем 4–6 различных каналов при взаимодействии с компанией. Во-вторых, развитие технологий позволило обеспечить техническую возможность интеграции разнородных систем. В-третьих, конкурентная среда вынуждает компании искать новые способы привлечения и удержания клиентов.

1.3 Ключевые компоненты омниканальной архитектуры

Омниканальная архитектура включает в себя несколько взаимосвязанных компонентов:

1. **Единая платформа управления взаимодействием** — центральный элемент архитектуры, обеспечивающий сбор, хранение и обработку данных о взаимодействиях с клиентами из всех каналов.
2. **Интеграционный слой** — совокупность программных интерфейсов (API) и middleware-решений, обеспечивающих обмен данными между различными системами.
3. **Система управления контентом** — обеспечивает единообразное представление информации на всех каналах.
4. **Аналитическая платформа** — собирает и анализирует данные о поведении клиентов, позволяя принимать обоснованные решения.
5. **Система управления взаимоотношениями с клиентами (CRM)** — хранит полную историю взаимодействия с каждым клиентом.

Особое значение в омниканальной архитектуре имеет качество данных. Поскольку информация поступает из множества источников, критически важным является обеспечение её непротиворечивости и актуальности.

1.4 Преимущества внедрения омниканальных решений

Внедрение омниканального подхода предоставляет компаниям ряд существенных преимуществ:

- **Повышение лояльности клиентов.** Бесшовный клиентский опыт способствует росту удовлетворённости и, как следствие, увеличению показателя удержания клиентов (retention rate).
- **Увеличение средней выручки на клиента.** Интеграция каналов открывает возможности для кросс-канальных продаж и персонализированных предложений.
- **Оптимизация операционных затрат.** Автоматизация процессов и единая аналитика позволяют сократить издержки на обслуживание клиентов.
- **Повышение эффективности маркетинга.** Консолидированные данные о клиентах обеспечивают более точное таргетирование и персонализацию коммуникаций.

Исследования показывают, что компании, успешно внедрившие омниканальную стратегию, демонстрируют на 10–15% более высокие показатели удовлетворённости клиентов и на 20–30% больший рост выручки по сравнению с компаниями, использующими мультиканальный подход.

1.5 Технологические аспекты реализации

Реализация омниканального подхода требует использования современных технологических решений. Ключевую роль играют облачные технологии, обеспечивающие масштабируемость и доступность систем. Микросервисная архитектура позволяет гибко наращивать функциональность и интегрировать новые каналы.

В области управления данными широкое применение находят технологии Big Data и системы класса Customer Data Platform (CDP), которые обеспечивают сбор и унификацию данных из различных источников. Машинное обучение используется для персонализации взаимодействий и прогнозирования поведения клиентов.

Безопасность данных является критически важным аспектом, особенно в контексте требований законодательства о защите персональных данных. Омниканальные решения должны обеспечивать соответствие требованиям Федерального закона № 152-ФЗ «О персональных данных» и других нормативных актов.

1.6 Отраслевые особенности применения

Омниканальные решения находят применение в различных отраслях экономики. В розничной торговле они позволяют объединить онлайн- и офлайн-каналы продаж, обеспечивая такие сценарии, как «клик-энд-коллект» (заказ онлайн с самовывозом в магазине) и возврат товаров, приобретённых онлайн, в физических точках продаж.

В банковском секторе омниканальность проявляется в интеграции мобильного банкинга, интернет-банка, отделений и контакт-центра. Клиент может начать оформление продукта в мобильном приложении и завершить его в отделении без повторного ввода данных.

В сфере телекоммуникаций омниканальный подход обеспечивает единое обслуживание клиентов через личный кабинет, мобильное приложение, чаты поддержки и центры продаж.

Выводы по главе 1

Омниканальные решения представляют собой современный подход к организации взаимодействия с клиентами, обеспечивающий бесшовную интеграцию всех каналов коммуникации. Внедрение такого подхода требует значительных инвестиций в технологии и организационные изменения, однако обеспечивает существенные конкурентные преимущества за счёт повышения качества клиентского опыта и эффективности бизнес-процессов.

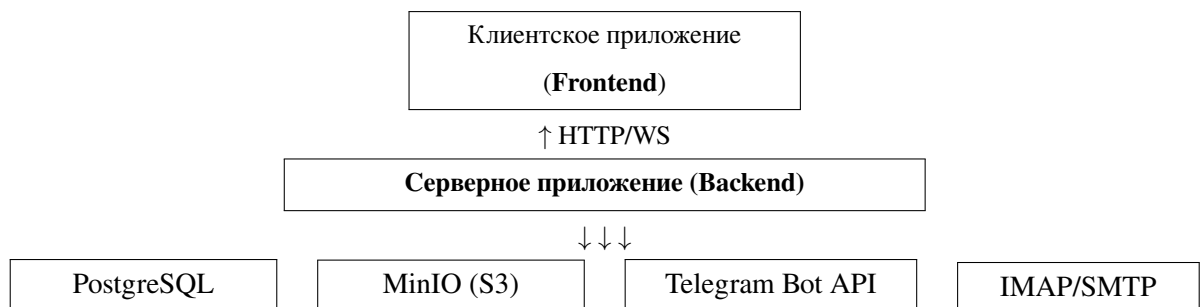
Глава 2

Архитектура программного комплекса омниканального решения

2.1 Общая архитектура системы

Разработанный программный комплекс представляет собой омниканальную платформу для взаимодействия с клиентами, объединяющую несколько каналов связи в едином интерфейсе. Система построена по принципу **Clean Architecture** (чистая архитектура) с монолитным развёртыванием и состоит из трёх основных уровней: клиентское приложение (фронтенд), серверное приложение (бэкенд) и внешние сервисы.

Общая структурная схема системы представлена ниже:



Основные компоненты системы развёртываются в среде контейнеризации Docker, что обеспечивает изолированность окружений и простоту масштабирования. Конфигурация взаимодействия компонентов задаётся через переменные окружения.

2.2 Серверная часть (Backend)

2.2.1 Архитектурный подход

Серверная часть реализована на языке **Python 3.14+** с использованием асинхронного веб-фреймворка **Litestar 2.x**. В основе архитектуры лежит принцип разделения ответственности (Clean Architecture), который предполагает чёткое разделение кода на три слоя:

- **Core (ядро)** — содержит доменные модели, бизнес-логику и порты (интерфейсы) для взаимодействия с внешними системами.

- **API** — предоставляет REST-эндпоинты и WebSocket для взаимодействия с клиентским приложением.
- **Adapters (адаптеры)** — реализуют порты ядра для конкретных технологий (PostgreSQL, Telegram, Email, MinIO).

Такая архитектура обеспечивает слабую связанность компонентов и возможность замены любой реализации без изменения бизнес-логики.

2.2.2 Схема слоёв бэкенда



2.2.3 Технологический стек бэкенда

- **Веб-фреймворк:** Litestar 2.x (асинхронный, поддержка OpenAPI, WebSocket)
- **ORM:** SQLAlchemy 2.x (асинхронный режим)
- **База данных:** PostgreSQL 17 (через asyncpg)
- **DI-контейнер:** ruq (внедрение зависимостей)
- **Аутентификация:** JWT (HS256) + bcrypt
- **Файловое хранилище:** MinIO (S3-совместимое объектное хранилище)
- **Telegram:** python-telegram-bot 22.x (long-polling)
- **Email:** IMAP (входящие), SMTP (исходящие)

2.2.4 API-эндпоинты

Серверная часть предоставляет следующие группы эндпоинтов:

- **Сообщения:** получение списка, просмотр, отправка ответа, загрузка файлов
- **Клиенты:** создание, просмотр, редактирование, поиск по каналу

- **Кураторы:** управление учётными записями операторов поддержки
- **Назначения:** привязка куратора к сообщению, передача между кураторами, история
- **Аутентификация:** регистрация, вход, получение текущего пользователя
- **WebSocket:** уведомления о новых сообщениях и переназначениях в реальном времени

Все REST-ответы возвращаются в формате JSON. При возникновении ошибок используется формат Problem Details (RFC 9457).

2.3 Клиентская часть (Frontend)

Клиентское приложение представляет собой одностраничное приложение (SPA), реализованное с использованием современного стека веб-технологий:

- **Фреймворк:** React 19
- **Язык:** TypeScript 5.8 (строгая типизация)
- **Сборщик:** Vite 6
- **Стилизация:** Tailwind CSS 3 (адаптивный дизайн, поддержка тёмной темы)

2.3.1 Структура пользовательского интерфейса

Интерфейс состоит из двух основных панелей:

- **Левая панель** (320 px): отображает список контактов с возможностью поиска, сортировки и фильтрации по каналу связи. Для каждого контакта отображается аватар, имя, последнее сообщение и количество непрочитанных сообщений.
- **Правая панель:** содержит область просмотра сообщений (история диалога) и форму ввода ответа. Форма ввода адаптируется под тип канала: для чатов — поле ввода с прикреплением файлов, для email — отдельная форма с полями «Кому», «Тема» и «Тело письма».

2.3.2 Взаимодействие с сервером

Обмен данными между клиентом и сервером осуществляется по протоколу HTTP (REST API) для основных операций и через WebSocket для получения уведомлений в реальном времени. При загрузке страницы устанавливается WebSocket-соединение с автоматическим переподключением при разрыве (с задержкой 3 секунды).

Аутентификация выполняется по JWT-токену, который сохраняется в `localStorage` браузера и передаётся в заголовке `Authorization: Bearer <token>` при каждом запросе.

2.4 Схема базы данных

Система использует реляционную базу данных PostgreSQL 17, содержащую следующие основные таблицы:

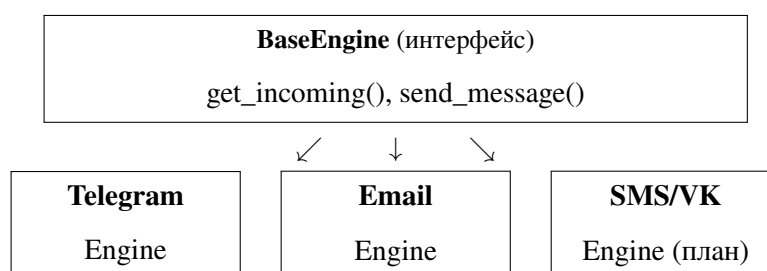
- **messages** — хранит все сообщения из всех каналов. Ключевые поля: идентификатор (формат `канал:оригинальный_id`), тип канала, идентификатор отправителя, содержимое, тип сообщения (текст, фото, документ и т.д.), идентификатор родительского сообщения (для тредов), назначенный куратор, временная метка.
- **senders** — отправители сообщений (физические или юридические лица). Содержит имя и связанные идентичности.
- **sender_identities** — привязка отправителя к конкретному каналу (например, Telegram ID или email-адрес).
- **clients** — клиенты, с которыми ведётся диалог. Содержит имя, телефон, email, ссылку на аватар, метаданные.
- **client_channels** — привязка клиента к каналам связи (канал + внешний идентификатор + имя пользователя).
- **curators** — операторы поддержки (кураторы). Содержит ФИО, логин, email, роль, статус, хеш пароля.
- **assignment_history** — история назначения кураторов на сообщения (кто, от кого, кому, причина, временная метка).

2.5 Модуль интеграции каналов связи

Ключевым компонентом системы является модуль интеграции каналов, реализованный с использованием паттерна **Абстрактная фабрика** (Abstract Factory). Каждый канал представлен отдельным классом-движком (Engine), реализующим единый интерфейс для получения и отправки сообщений.

Таблица 2.1: Сводная таблица базы данных

Таблица	Кол-во полей	Назначение
messages	11	Все сообщения из всех каналов
senders	3	Отправители сообщений
sender_identities	4	Привязка отправителей к каналам
clients	7	Клиенты системы
client_channels	5	Каналы клиентов
curators	9	Операторы поддержки
assignment_history	7	История назначений



2.5.1 Модуль Telegram

Интеграция с мессенджером Telegram реализована через официальное API с использованием библиотеки `python-telegram-bot`. Модуль работает в режиме long-polling, периодически опрашивая сервер Telegram на наличие новых сообщений. Поддерживаются следующие типы контента:

- Текстовые сообщения
- Фотографии (загружаются в MinIO, фронтенду передаётся presigned-ссылка)
- Документы (аналогичная обработка)
- Стикеры (поддерживаются форматы .webp, .webm, .tgs)
- Анимации (GIF)
- Видео, аудио и голосовые сообщения

При поступлении нового сообщения модуль автоматически создаёт или обновляет запись клиента в базе данных, сохраняет сообщение и уведомляет подключённых операторов через WebSocket.

2.5.2 Модуль Email

Интеграция с электронной почтой реализована по протоколам IMAP (для получения входящих сообщений) и SMTP (для отправки ответов). Поддерживаются как текстовые, так и HTML-письма. На фронтенде HTML-часть письма отображается в iframe-контейнере.

Цикл обработки email-сообщений:

1. Периодический опрос IMAP-сервера на предмет новых непрочитанных писем.
2. Извлечение текстового и HTML-содержимого, декодирование темы и заголовков.
3. Сохранение в базе данных в формате, едином для всех каналов.
4. Отправка ответа через SMTP с сохранением темы письма (префикс «Re:»).

2.5.3 Планируемые каналы

Архитектура системы предусматривает возможность подключения дополнительных каналов без изменения существующего кода. В настоящий момент в кодовой базе предусмотрена поддержка (включена в перечисление типов каналов, но не реализована):

- **SMS** — планируется через интеграцию с SMS-шлюзом
- **VK** — планируется через VK API

Подключение нового канала сводится к реализации класса-движка, наследующего интерфейс BaseEngine, и регистрации его фабрики в общем реестре.

2.6 Взаимодействие компонентов

2.6.1 Получение входящего сообщения

Процесс получения и обработки входящего сообщения включает следующие шаги:

1. **Пинг канала:** компонент PollingOrchestrator запускает асинхронную задачу для каждого активного движка канала.
2. **Получение:** движок канала (Telegram, Email) получает новое сообщение от внешнего API.
3. **Создание/обновление клиента:** система проверяет, существует ли клиент с указанным внешним идентификатором. Если нет — создаётся новый клиент.

4. **Сохранение:** сообщение сохраняется в таблицу `messages` через репозиторий.
5. **Уведомление:** через `WebSocket` всем подключённым операторам отправляется событие `new_message`.
6. **Отображение:** фронтенд получает событие и добавляет сообщение в интерфейс соответствующего оператора.

2.6.2 Отправка ответа

1. Оператор вводит текст ответа в интерфейсе и нажимает «Отправить».
2. Фронтенд отправляет `POST`-запрос на `/api/messages/reply`.
3. Бэкенд создаёт объект сообщения-ответа со ссылкой на родительское сообщение (`parent_id`).
4. Движок соответствующего канала отправляет сообщение через внешнее API (Telegram — в чат, Email — на почтовый адрес).
5. Ответ сохраняется в базу данных и транслируется через `WebSocket`.

2.6.3 Назначение куратора

1. Оператор выбирает сообщение и назначает ответственного куратора (себя или другого оператора).
2. Фронтенд отправляет запрос на `POST /api/messages/{id}/assign`.
3. Бэкенд обновляет поле `curator_id` в записи сообщения и создаёт запись в `assignment_history`.
4. Система уведомляет всех операторов через `WebSocket` о назначении.

Выводы по главе 2

Разработанный программный комплекс реализует архитектуру, обеспечивающую надёжную и масштабируемую работу омниканальной платформы. Использование Clean Architecture позволяет изолировать бизнес-логику от технических деталей реализации, что упрощает поддержку и развитие системы. Модульная архитектура интеграции каналов обеспечивает возможность подключения новых каналов связи с минимальными трудозатратами. Применение современных технологий (Python 3.14, Litestar, React 19, PostgreSQL 17, WebSocket) гарантирует высокую производительность и отзывчивость интерфейса.

Глава 3

Реализация программного комплекса

В данной главе рассматриваются ключевые аспекты реализации системы, включая доменную модель, бизнес-логику, адаптеры каналов связи, репозитории для работы с базой данных, а также конфигурацию и развёртывание.

3.1 Доменная модель

Центральным элементом предметной области является класс `Message`, представляющий сообщение из любого канала связи. Модель реализована в виде неизменяемого `dataclass`-а с использованием `slots=True` для оптимизации потребления памяти:

```
1 @dataclass(slots=True)
2 class Message:
3     id: str
4     channel: str
5     sender_id: str
6     content: str
7     timestamp: datetime
8     metadata: Dict[str, Any]
9     message_type: MessageType = MessageType.TEXT
10    recipient: Optional[str] = None
11    subject: Optional[str] = None
12    parent_id: Optional[str] = None
13    curator_id: Optional[str] = None
14
15    def to_dict(self) -> Dict[str, Any]:
16        return {
17            "id": self.id,
18            "channel": self.channel,
19            "sender_id": self.sender_id,
20            "content": self.content,
```

```

21         "timestamp": self.timestamp.isoformat(),
22         "message_type": self.message_type.value,
23         "metadata": self.metadata,
24         "recipient": self.recipient,
25         "subject": self.subject,
26         "parent_id": self.parent_id,
27         "curator_id": self.curator_id,
28     }

```

Поле `channel` принимает одно из значений перечисления `ChannelType`, которое определяет поддерживаемые каналы связи:

```

1 class ChannelType(StrEnum):
2     TELEGRAM = "telegram"
3     EMAIL = "email"
4     SMS = "sms"
5     VK = "vk"

```

Тип вложенного контента (текст, фото, документ, стикер и т.д.) определяется перечислением `MessageType`. Такой подход обеспечивает единообразное представление разнородных сообщений из всех каналов, что является ключевым требованием омниканальной системы.

3.2 Порты (интерфейсы)

Для обеспечения слабой связанности компонентов каждый модуль зависит от абстракции (порта), а не от конкретной реализации. Интерфейс движка канала связи определён следующим образом:

```

1 class MessageEngine(ABC):
2     @abstractmethod
3     async def start(self) -> None:
4         ...
5
6     @abstractmethod
7     async def send_message(
8         self, recipient: str, content: str, **kwargs
9     ) -> str:

```

```

10         ...
11
12     @abstractmethod
13     def get_incoming(self) -> AsyncIterator[Message]:
14         ...
15
16     @property
17     @abstractmethod
18     def channel_type(self) -> str:
19         ...

```

Метод `get_incoming()` возвращает асинхронный итератор, что позволяет эффективно обрабатывать поток входящих сообщений без блокировки основного потока выполнения. Метод `send_message()` возвращает идентификатор отправленного сообщения для последующей синхронизации состояния.

Аналогичным образом определены порты для репозитория сообщений, клиентов, кураторов и шлюза базы данных.

3.3 Бизнес-логика

3.3.1 Сервис сообщений

`MessageService` реализует сценарии использования, связанные с управлением сообщениями. Основные операции делегируются репозиторию через интерфейс `MessageRepository`:

```

1 class MessageService:
2     def __init__(self, repository: MessageRepository):
3         self._repository = repository
4
5     async def save_message(self, message: Message) -> None:
6         await self._repository.save_message(message)
7
8     async def get_messages(
9         self,
10        channel: Optional[str] = None,
11        sender_id: Optional[str] = None,
12        curator_id: Optional[str] = None,

```

```

13         limit: int = 100,
14         offset: int = 0,
15     ) -> List[Message]:
16         return await self._repository.get_messages(
17             channel=channel, sender_id=sender_id,
18             curator_id=curator_id, limit=limit, offset=offset,
19         )
20
21     async def reply_to_message(
22         self, message_id: str, content: str
23     ) -> Message:
24         original = await
25             self._repository.get_message_by_id(message_id)
26         if not original:
27             raise MessageNotFoundError(message_id)
28         return original

```

Метод `reply_to_message()` проверяет существование исходного сообщения и в случае его отсутствия генерирует исключение `MessageNotFoundError`, которое в дальнейшем преобразуется в HTTP-ответ с кодом 404 в формате Problem Details (RFC 9457).

3.3.2 Оркестратор опроса каналов

`PollingOrchestrator` отвечает за непрерывный сбор входящих сообщений из всех зарегистрированных каналов. Класс реализует паттерн «Наблюдатель» (Observer) и использует асинхронные задачи для параллельного опроса движков:

```

1 class PollingOrchestrator(AbstractPollingService):
2     def __init__(self, message_service, client_service):
3         self._message_service = message_service
4         self._client_service = client_service
5         self._engines: List[MessageEngine] = []
6         self._tasks: List[asyncio.Task] = []
7
8     def register_engine(self, engine: MessageEngine) -> None:
9         self._engines.append(engine)
10
11     async def start_polling(self) -> None:

```

```

12         self._running = True
13         for engine in self._engines:
14             await engine.start()
15             task = asyncio.create_task(
16                 self._poll_engine(engine)
17             )
18             self._tasks.append(task)
19
20     async def send_reply(self, channel, recipient,
21                         content, **kwargs) -> str | None:
22         for engine in self._engines:
23             if engine.channel_type == channel:
24                 return await engine.send_message(
25                     recipient=recipient, content=content, **kwargs
26                 )
27         return None

```

Основной цикл обработки реализован в методе `_poll_engine()`. Для каждого полученного сообщения система автоматически выполняет следующие действия:

1. Извлечение информации об отправителе из метаданных канала.
2. Создание или обновление записи клиента через `ClientService`.
3. Сохранение сообщения в базу данных.
4. Трансляция события `new_message` всем подключённым операторам через `WebSocket`.

Извлечение информации об отправителе учитывает особенности каждого канала: для Telegram извлекаются `username` и `first_name`, для email — имя и адрес из заголовка «From».

3.4 Адаптеры каналов связи

3.4.1 Telegram

Адаптер Telegram реализован с использованием библиотеки `python-telegram-bot`. Класс `TelegramEngine` обрабатывает различные типы вложений, загружая медиафайлы в объектное хранилище `MinIO` и формируя `presigned`-ссылки для доступа со стороны фронтенда.

Метод отправки сообщения поддерживает как текстовые сообщения, так и сообщения

с вложениями:

```
1 async def send_message(  
2     self, recipient: str, content: str, **kwargs  
3 ) -> str:  
4     if "file_url" in kwargs:  
5         with aiohttp.ClientSession() as session:  
6             async with session.get(kwargs["file_url"]) as resp:  
7                 file_data = await resp.read()  
8                 file = InputFile(BytesIO(file_data),  
9                                 filename=kwargs.get("filename", "file"))  
10                sent = await self._bot.send_document(  
11                    chat_id=recipient, document=file,  
12                    caption=content or None  
13                )  
14            else:  
15                sent = await self._bot.send_message(  
16                    chat_id=recipient, text=content  
17                )  
18            return str(sent.id)
```

Для загрузки вложений в S3-хранилище реализованы вспомогательные методы:

```
1 async def _upload_photo_to_s3(  
2     self, photo: PhotoSize, message_id: str  
3 ) -> Optional[str]:  
4     file = await photo.get_file()  
5     file_bytes = BytesIO()  
6     await file.download_to_memory(file_bytes)  
7     file_bytes.seek(0)  
8     url = await self._s3.upload_file(  
9         bucket="attachments",  
10        key=f"{message_id}/photo.jpg",  
11        data=file_bytes.getvalue(),  
12    )  
13    return url
```

3.4.2 Email

Адаптер электронной почты использует встроенные библиотеки Python `imaplib` (получение) и `smtplib` (отправка), работая через `asyncio.run_in_executor()` для совместимости с асинхронной архитектурой. Цикл опроса IMAP-сервера:

```
1 async def get_incoming(self) -> AsyncIterator[Message]:
2     while self._running:
3         messages = await asyncio.get_event_loop().run_in_executor(
4             None, self._fetch_new_messages
5         )
6         for msg in messages:
7             yield msg
8         await asyncio.sleep(self._poll_interval)
```

Извлечение содержимого письма обрабатывает MIME-структуру, поддерживая как текстовые, так и HTML-части:

```
1 def _extract_body(self, msg) -> tuple[str, str | None]:
2     text_body = ""
3     html_body = None
4     if msg.is_multipart():
5         for part in msg.walk():
6             content_type = part.get_content_type()
7             if content_type == "text/plain":
8                 text_body += self._decode_part(part)
9             elif content_type == "text/html":
10                 html_body = self._decode_part(part)
11     else:
12         text_body = self._decode_part(msg)
13     return text_body, html_body
```

3.4.3 Фабрика движков

Для создания экземпляров движков используется паттерн «Абстрактная фабрика». Каждый тип канала имеет свою фабрику, которая регистрируется в общем реестре:

```
1 class TelegramEngineFactory(EngineFactory):
2     def create_engine(self, config: dict) -> MessageEngine:
3         return TelegramEngine(token=config["token"], s3=self._s3)
```



```

4
5     def get_supported_channel(self) -> str:
6         return ChannelType.TELEGRAM
7
8
9 class EngineAbstractFactory:
10     def __init__(self):
11         self._factories: Dict[str, EngineFactory] = {}
12
13     def register_factory(self, factory: EngineFactory) -> None:
14         channel = factory.get_supported_channel()
15         self._factories[channel] = factory
16
17     def create_engine(self, channel_type, config) ->
18         MessageEngine:
19         if channel_type not in self._factories:
20             raise ValueError(
21                 f"No factory for channel: {channel_type}"
22             )
23         return self._factories[channel_type].create_engine(config)

```

Такой подход позволяет добавлять новые каналы связи простой реализацией нового движка и его фабрики, без модификации существующего кода.

3.5 Репозиторий и ORM

Для работы с базой данных используется SQLAlchemy 2.x в асинхронном режиме. Модель сообщения в ORM представляет собой отображение на таблицу messages:

```

1 class MessageModel(Base):
2     __tablename__ = "messages"
3
4     id = Column(String, primary_key=True)
5     channel = Column(String(50), nullable=False, index=True)
6     sender_id = Column(String, ForeignKey("senders.id",
7                                         ondelete="SET NULL"),
8                     nullable=True, index=True)

```

```

9     content = Column(Text, nullable=False)
10    message_type = Column(String(20), nullable=False,
11                           server_default="text")
12    subject = Column(String(500), nullable=True)
13    parent_id = Column(String, nullable=True, index=True)
14    curator_id = Column(String, nullable=True, index=True)
15    timestamp = Column(DateTime(timezone=True), nullable=False)
16    metadata_ = Column("metadata", JSON, nullable=True)
17    created_at = Column(DateTime(timezone=True),
18                       server_default=func.now())

```

Репозиторий реализует интерфейс `MessageRepository`, инкапсулируя детали выполнения SQL-запросов. Методы репозитория работают с доменными объектами, а не с ORM-моделями, что обеспечивает независимость бизнес-логики от выбранной СУБД.

3.6 Контейнер внедрения зависимостей

Для связывания компонентов используется DI-контейнер `punq`. Все зависимости регистрируются в единственной точке конфигурации:

```

1 def configure_container() -> punq.Container:
2     container = punq.Container()
3
4     container.register(EngineAbstractFactory,
5                       instance=EngineAbstractFactory())
6     container.register(S3Interface, instance=MinioGateway(...))
7     container.register(DatabaseGateway,
8                       instance=PostgresDatabaseGateway(...))
9     container.register(MessageRepository,
10                       instance=PostgresMessageRepository(...))
11    container.register(MessageService)
12    container.register(ClientService)
13    container.register(AbstractPollingService,
14                      instance=PollingOrchestrator(...))
15    container.register(AuthService)
16    container.register(ValidationService)
17

```

```
18         return container
```

Контейнер обеспечивает автоматическое разрешение зависимостей: если сервис требует `MessageRepository`, контейнер внедряет соответствующий экземпляр без необходимости ручного конструирования.

Точка входа в приложение — функция создания приложения `Litestar`:

```
1 @asynccontextmanager
2 async def lifespan(app: Litestar):
3     container = configure_container()
4     db = container.resolve(DatabaseGateway)
5     await db.init()
6     polling = container.resolve(AbstractPollingService)
7     await polling.start_polling()
8     yield
9     await polling.stop_polling()
10    await db.close()
11
12
13 app = Litestar(
14     route_handlers=[...],
15     lifespan=[lifespan],
16     websocket_handler=websocket_handler,
17     on_app_init=[...],
18 )
```

3.7 Конфигурация и развёртывание

Система развёртывается с использованием `Docker Compose`, что обеспечивает изолированное окружение для всех компонентов:

```
services:
  postgres:
    image: postgres:17
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
```

```

    POSTGRES_DB: omnichannel
healthcheck:
    test: ["CMD-SHELL", "pg_isready -U user -d omnichannel"]
volumes:
    - postgres_data:/var/lib/postgresql/data

minio:
    image: minio/minio
    command: server /data --console-address ":9001"
    environment:
        MINIO_ROOT_USER: minioadmin
        MINIO_ROOT_PASSWORD: minioadmin
    volumes:
        - minio_data:/data

app:
    build: .
    env_file: .env
    ports:
        - "8000:8000"
    depends_on:
        postgres: { condition: service_healthy }

```

Конфигурация приложения задаётся через переменные окружения (файл `.env`), что позволяет гибко настраивать параметры подключения к базам данных, токены каналов и настройки безопасности без изменения кода.

3.8 Тестирование

Покрытие кода модульными и интеграционными тестами реализовано с использованием `pytest` совместно с `pytest-asyncio` для асинхронных тестов. Пример теста сервиса опроса каналов:

```

1 async def test_poll_engine_saves_messages(
2     mock_engine, mock_message_service, mock_client_service
3 ):
4     orchestrator = PollingOrchestrator(

```

```

5         message_service=mock_message_service,
6         client_service=mock_client_service,
7     )
8     orchestrator.register_engine(mock_engine)
9
10    mock_engine.set_messages([...])
11    await orchestrator._poll_engine(mock_engine)
12
13    assert mock_message_service.save_message.called

```

Тесты репозитория используют Docker-контейнер с PostgreSQL (через `pytest-postgresql` или тестовую БД), что гарантирует корректность SQL-запросов и целостность данных. На момент написания отчёта тестовое покрытие включает тестирование всех сервисов, адаптеров, API-маршрутов и обработчиков исключений.

Выводы по главе 3

В ходе реализации программного комплекса были применены современные подходы к разработке: Clean Architecture для разделения ответственности, паттерны Abstract Factory и Observer для гибкой интеграции каналов, асинхронное программирование для обеспечения производительности, а также Dependency Injection для связывания компонентов. Применение Docker Compose упрощает развёртывание и обеспечивает воспроизводимость окружения. Тестовое покрытие подтверждает корректность реализации ключевых сценариев использования системы.

На основе проведённой работы можно сделать вывод, что разработанная архитектура соответствует всем требованиям, предъявляемым к современным омниканальным решениям: модульность, масштабируемость, расширяемость и надёжность.

Заключение

В ходе прохождения производственной практики я успешно выполнил работы по проектированию и разработке омниканальной платформы для центра обучения, обеспечивающей приём и обработку обращений из различных каналов связи в едином интерфейсе куратора.

Была спроектирована архитектура системы, основанная на принципах Clean Architecture. Определены три уровня: Core (доменные модели и бизнес-логика), API (REST-эндпоинты и WebSocket) и Adapters (реализации PostgreSQL, Telegram, Email, MinIO). Разработанная архитектура обеспечивает слабую связанность компонентов и позволяет в дальнейшем подключать новые каналы связи без изменения существующего кода.

Разработана схема базы данных PostgreSQL, включающая семь таблиц для хранения информации о сообщениях, отправителях, клиентах, каналах клиентов, кураторах и истории назначений. При проектировании использованы внешние ключи, индексы и ограничения для обеспечения целостности данных.

Особое внимание было уделено организации процесса разработки и развёртывания приложения. Для локальной разработки использована контейнеризация Docker и Docker Compose, включающая PostgreSQL 17, MinIO и серверное приложение. Настроены pre-commit хуки, линтеры (ruff) и система статической типизации (mypy). Реализовано тестовое покрытие ключевых сервисов, адаптеров и API-маршрутов с использованием pytest и pytest-asyncio.

В результате прохождения практики были закреплены навыки проектирования архитектуры информационных систем, разработки серверных приложений на Python с использованием асинхронного веб-фреймворка Litestar, проектирования реляционных баз данных PostgreSQL, интеграции внешних API (Telegram Bot API, IMAP/SMTP), разработки клиентских приложений на React и организации развёртывания современных веб-приложений с помощью Docker.

В перспективе система может быть расширена дополнительными каналами связи (SMS, VK), интеграцией с образовательной платформой GetCourse, а также дополнена функциональностью аналитики и отчётности.

Приложение А

Исходный код проекта

Исходный код разработанного в ходе практики программного комплекса доступен в публичном репозитории на GitHub по адресу:

<https://github.com/ProudRykar/PP-2026>

Структура репозитория включает следующие основные директории и файлы:

- `app/` — серверная часть на Python с использованием фреймворка Litestar
 - `core/` — доменная модель, бизнес-логика и порты (интерфейсы)
 - `api/` — REST-контроллеры, DTO-схемы, обработка исключений
 - `adapters/` — реализации адаптеров (Telegram, Email, PostgreSQL, MinIO)
- `frontend/` — клиентское приложение на React 19 с TypeScript
- `tests/` — модульные и интеграционные тесты (pytest, pytest-asyncio)
- `docker-compose.yml` — конфигурация развёртывания (PostgreSQL, MinIO, приложение)
- `pyproject.toml` — зависимости и настройки инструментов разработки